

Projective algorithm

In this example we use the [pyALF \(https://git.physik.uni-wuerzburg.de/ALF/pyALF\)](https://git.physik.uni-wuerzburg.de/ALF/pyALF) interface to run ALF's projective algorithm with the Mz choice of Hubbard Stratonovich transformation on a 4-site ring.

The projective approach is the method of choice if one is interested in ground-state properties. The starting point is a pair of trial wave functions, $|\Psi_{T,L/R}\rangle$, that are not orthogonal to the ground state $|\Psi_0\rangle$: $\langle\Psi_0|\Psi_{T,L/R}\rangle \neq 0$. The ground-state expectation value of any observable $\hat{O}$ can then be computed by propagation along the imaginary time axis: $\langle\Psi_0|\hat{O}|\Psi_0\rangle = \lim_{\theta \rightarrow \infty} \frac{\langle\Psi_0|\Psi_{T,L}\rangle \langle\Psi_{T,L}|\hat{O}|\Psi_{T,L}\rangle e^{-(\beta-\tau)\langle H\rangle}}{\langle\Psi_{T,L}|\Psi_{T,L}\rangle e^{-(2\theta+\beta)\langle H\rangle} \langle\Psi_{T,R}|\Psi_{T,R}\rangle}$, where β defines the imaginary time range where observables (time displaced and equal time) are measured and τ varies from 0 to β in the calculation of time-displace observables. For further details, see Sec. 3 of [ALF documentation \(https://git.physik.uni-wuerzburg.de/ALF/ALF_code/-/blob/master/Documentation/ALF_v1.0.pdf\)](https://git.physik.uni-wuerzburg.de/ALF/ALF_code/-/blob/master/Documentation/ALF_v1.0.pdf).

1. Import `Simulation` class from the `py_alf` python module, which provides the interface with ALF, as well as numerical and plotting packages:

In [1]:

```
from py_alf import Simulation      # Interface with ALF
#
import numpy as np                # Numerical library
from scipy.optimize import curve_fit # Numerical library
import matplotlib.pyplot as plt   # Plotting library
```

2. Create instances of `Simulation`, specifying the necessary parameters, in particular the `Projector` to `True`:

In [2]:

```
sims = []                                # Vector of Simulation instances
print('Theta values used:')
for theta in [5, 10, 20]:                # Values of Theta
    print(theta)
    sim = Simulation(
        'Hubbard',                        # Hamiltonian
        {                                  # Model and simulation parameters for each Simulation instance
            'Model': 'Hubbard',            # Base model
            'Lattice_type': 'N_leg_ladder', # Lattice type
            'L1': 4,                       # Lattice length in the first unit vector direction
            'L2': 1,                       # Lattice length in the second unit vector direction
            'Checkerboard': False,          # Whether checkerboard decomposition is used or not
            'Symm': True,                   # Whether symmetrization takes place
            'Projector': True,              # Whether to use the projective algorithm
            'Theta': theta,                 # Projector parameter
            'ham_T': 1.0,                   # Hopping parameter
            'ham_U': 4.0,                   # Hubbard interaction
            'ham_Tperp': 0.0,                # For bilayer systems
            'beta': 1.0,                    # Inverse temperature
            'Ltau': 0,                      # '1' for time-displaced Green functions; '0' otherwise
            'NSweep': 400,                  # Number of sweeps
            'NBin': 10,                    # Number of bins
            'Dtau': 0.05,                   # Only dtau varies between simulations, Ltrot=beta/Dtau
            'Mz': True,                     # If true, sets the M_z-Hubbard model: Nf=2, N_sum=1,
            },                               # HS field couples to z-component of magnetization
        alf_dir='~/Programs/ALF',          # Local ALF copy, if present
    )
    sims.append(sim)
```

Theta values used:

5
10
20

3. Compile ALF, downloading it first if not found locally. This may take a few minutes:

In [3]:

```
sims[0].compile()                      # Compilation needs to be performed only once
```

Compiling ALF... Done.

4. Perform the simulations, as specified in each element of `sim`:

In [4]:

```
for i, sim in enumerate(sims):
    sim.run()                                # Perform the actual simulation in ALF
```

Prepare directory "/home/stafusa/ALF/pyALF/Hubbard_N_leg_ladder_L1=4_L2=1_Checkerboard=False_Symm=True_Projector=True_Theta=5_T=1.0_U=4.0_Tperp=0.0_beta=1.0_Dtau=0.05_Mz=True" for Monte Carlo run.

Create new directory.

Run /home/stafusa/Programs/ALF/Prog/Hubbard.out

Prepare directory "/home/stafusa/ALF/pyALF/Hubbard_N_leg_ladder_L1=4_L2=1_Checkerboard=False_Symm=True_Projector=True_Theta=10_T=1.0_U=4.0_Tperp=0.0_beta=1.0_Dtau=0.05_Mz=True" for Monte Carlo run.

Create new directory.

Run /home/stafusa/Programs/ALF/Prog/Hubbard.out

Prepare directory "/home/stafusa/ALF/pyALF/Hubbard_N_leg_ladder_L1=4_L2=1_Checkerboard=False_Symm=True_Projector=True_Theta=20_T=1.0_U=4.0_Tperp=0.0_beta=1.0_Dtau=0.05_Mz=True" for Monte Carlo run.

Create new directory.

Run /home/stafusa/Programs/ALF/Prog/Hubbard.out

5. Calculate the internal energies:

In [6]:

```
%%capture
ener = np.empty((len(sims), 2))           # Matrix for storing energy values
thetas = np.empty((len(sims),))          # Matrix for Thetas values, for plotting
for i, sim in enumerate(sims):
    print(sim.sim_dir)                    # Directory containing the simulation output
    sim.analysis()                         # Perform default analysis
    thetas[i] = sim.sim_dict['Theta']      # Store Theta value
    ener[i] = sim.get_obs(['Ener_scalJ'])['Ener_scalJ']['obs'] # Store internal energy
```

Where the *cell magic* %%capture suppresses the output of sim.analysis(), which lists the data directories and observables.

In [7]:

```
print('For Theta values', thetas, 'the measured energies are:\n', ener)
```

For Theta values [5. 10. 20.] the measured energies are:

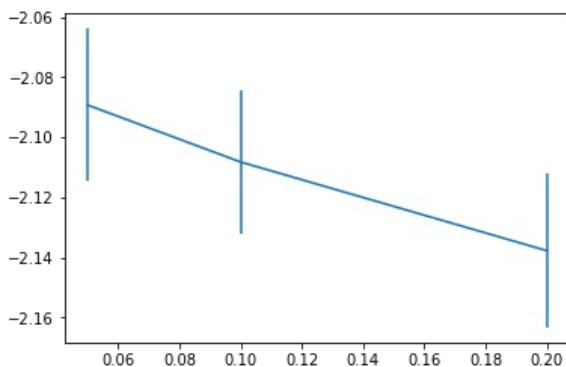
```
[[-2.137697  0.025552]
 [-2.108235  0.023964]
 [-2.089294  0.025426]]
```

In [8]:

```
plt.errorbar(1/thetas, ener[:, 0], ener[:, 1])
```

Out[8]:

<ErrorbarContainer object of 3 artists>



Exercises

1. [TO BE FLESHED OUT] Kondo phase transition. References:

<https://journals.aps.org/prb/abstract/10.1103/PhysRevB.63.155114> (<https://journals.aps.org/prb/abstract/10.1103/PhysRevB.63.155114>)
<https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.83.796> (<https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.83.796>)